Python essencial – consulta rápida 1 · Bases

01 VARIÁVEIS E NÚMEROS

cap. 4 e 5

`idade = 23`	Criar uma variável; o tipo não se declara.
`type(idade)`	`int`, `float`, `str`, `bool` ou `NoneType`.
`int("42")  float("3.5")  str(10)`	Converter entre tipos.
`7 / 2  7 // 2  7 % 2`	3.5 · 3 (divisão inteira) · 1 (resto).
`2 ** 10  round(3.14159, 2)`	Potência (1024) · arredondar (3.14).

02 STRINGS (TEXTO)

cap. 6

`s = "Python"`	Texto entre aspas simples ou duplas.
`s[0]  s[-1]  s[0:3]  s[::-1]`	P · n · Pyt · texto invertido.
`len(s)`	Número de caracteres (6).
`s.upper()  s.lower()  s.strip()`	Maiúsculas · minúsculas · sem espaços.
`s.replace("P", "J")`	Substituir; devolve um texto novo.
`"a,b,c".split(",")  "-".join(lista)`	Texto → lista · lista → texto.
`"th" in s`	Contém? → `True`.

03 ENTRADA E SAÍDA

cap. 7 e 9

`print("a", "b", sep="-", end="!")`	Mostra `a-b!`; `sep` e `end` são opcionais.
`nome = input("Nome: ")`	Lê do teclado: devolve sempre `str`.
`idade = int(input("Idade: "))`	Converter logo para número.
`f"{nome} tem {idade} anos"`	f-string: valores dentro do texto.
`f"{preco:.2f} €"  f"{0.25:.0%}"`	Duas casas decimais · 25%.
`f"{n:>5}"  f"{n:,}"`	Alinhar à direita · milhares.

04 OPERADORES

cap. 8

`==  !=  <  <=  >  >=`	Comparar → `True` ou `False`.
`and  or  not`	Combinar condições.
`x in lista  x not in lista`	Pertença.
`x is None`	Identidade; usa-se para `None`.
`x += 1  x *= 2`	Atualizar a própria variável.

05 DECISÕES

cap. 10

`if nota >= 10:`
`  print("Aprovado")`
`elif nota >= 8:`
`  print("Oral")`
`else:`
`  print("Reprovado")`
	Corre só o primeiro bloco verdadeiro. A indentação (4 espaços) define cada bloco.

`estado = "maior" if idade >= 18 else "menor"`	Expressão condicional numa linha.

`match comando:`
`  case "sair" | "q":`
`  terminar()`
`  case _:`
`  print("?")`
	Escolha múltipla (Python 3.10+). O `_` apanha os restantes casos.

06 CICLOS

cap. 11

`for i in range(5):`	i = 0, 1, 2, 3, 4 (o 5 não entra).
`range(1, 11, 2)`	1, 3, 5, 7, 9: início, fim, passo.
`for i, v in enumerate(lista):`	Posição e valor de cada elemento.
`for a, b in zip(nomes, notas):`	Percorrer duas listas em par.
`while saldo > 0:`	Repete enquanto a condição for verdadeira.
`break  continue`	Sair do ciclo · saltar para a volta seguinte.

07 LISTAS

cap. 12

`nums = [5, 3, 8]`	Ordenada, alterável, aceita repetidos.
`nums[0]` `nums[-1]` `nums[1:3]`	Primeiro · último · posições 1 e 2.
`nums.append(9)` `nums.insert(0, 7)`	Juntar no fim · na posição 0.
`nums.extend([1, 2])`	Juntar vários elementos.
`nums.remove(3)` `nums.pop()`	Tirar por valor · tirar o último.
`nums.sort()` `sorted(nums)`	Ordena a própria lista · devolve uma nova.
`len(nums)` `sum(nums)` `min(nums)` `max(nums)`	Tamanho e resumos.
`nums.index(8)` `nums.count(5)`	Posição de um valor · quantas vezes aparece.
`copia = nums.copy()`	Cópia independente. `b = a` não copia.
`[x ** 2 for x in nums if x % 2 == 0]`	List comprehension: transformar e filtrar numa linha.

08 TUPLOS

cap. 13

`t = (1, 2)` `um = (5,)`	Ordenado e imutável. Com um só elemento, leva vírgula.
`x, y = t`	Desempacotar.
`a, b = b, a`	Trocar dois valores.
`return minimo, maximo`	Uma função devolve vários valores num tuplo.

09 DICIONÁRIOS

cap. 14

`d = {"nome": "Ana", "idade": 23}`	Pares chave → valor.
`d["nome"]`	Ler; `KeyError` se a chave não existir.
`d.get("email", "–")`	Ler com valor por omissão.
`d["idade"] = 24`	Alterar ou acrescentar.
`del d["idade"]` `d.pop("idade")`	Remover uma chave.
`"nome" in d`	A chave existe?
`for chave, valor in d.items():`	Percorrer os pares. Há também `keys()` e `values()`.
`{n: len(n) for n in nomes}`	Dictionary comprehension.

10 CONJUNTOS

cap. 15

`s = {1, 2, 3}` `vazio = set()`	Sem repetidos nem ordem. `{}` é um dicionário.
`s.add(4)` `s.discard(9)`	Juntar · tirar (sem erro se não existir).
`a | b` `a & b` `a - b`	União · interseção · diferença.
`set(lista)`	Remover repetidos de uma lista.

Que estrutura usar?

Estrutura	Ordem	Alterável	Repetidos	Exemplo
lista	sim	sim	sim	`[1, 2]`
tuplo	sim	não	sim	`(1, 2)`
dicionário	de inserção	sim	chaves não	`{"a": 1}`
conjunto	não	sim	não	`{1, 2}`

11 FUNÇÕES

cap. 18

`def area(base, altura=1):`
`  """Área de um retângulo."""`
`  return base * altura`
	Parâmetros, valor por omissão, docstring e `return`.

`area(3)  area(3, altura=2)`	Chamar: argumento posicional · nomeado.
`def soma(*nums):`	Número variável de argumentos (tuplo).
`def config(**opcoes):`	Argumentos nomeados livres (dicionário).
`minimo, maximo = estatisticas(dados)`	Receber vários valores devolvidos.
`global contador`	Alterar uma variável global. Evita: prefere devolver valores.

`def fatorial(n):`
`  if n <= 1:`
`  return 1`
`  return n * fatorial(n - 1)`
	Recursão: a função chama-se a si própria e precisa de um caso base.

12 EXPRESSÕES LAMBDA

cap. 19

`dobro = lambda x: x * 2`	Função anónima de uma só expressão.
`sorted(pessoas, key=lambda p: p["idade"])`	Ordenar por um critério.
`list(map(lambda x: x * 2, nums))`	Aplicar a todos os elementos.
`list(filter(lambda x: x > 0, nums))`	Manter os que cumprem a condição.

13 EXCEÇÕES

cap. 16

`try:`
`  n = int(input("Número: "))`
`except ValueError:`
`  print("Não é um número.")`
`else:`
`  print("Lido:", n)`
`finally:`
`  print("Fim.")`
	`except`: corre se houver erro.
	`else`: corre se não houver.
	`finally`: corre sempre.

`except (ValueError, ZeroDivisionError) as e:`	Vários tipos; e guarda a mensagem.
`raise ValueError("Idade negativa")`	Lançar um erro de propósito.
`NameError  TypeError` `KeyError  IndexError`	Nome desconhecido · tipo errado · chave inexistente · posição fora da lista.

14 FICHEIROS

cap. 17

`with open("notas.txt", "w",`
`  encoding="utf-8") as f:`
`  f.write("Ana;18\n")`
	Escrever. O `with` fecha o ficheiro sozinho.

`"r"  "w"  "a"`	Ler · escrever (apaga o que havia) · acrescentar no fim.

`with open("notas.txt",`
`  encoding="utf-8") as f:`
`  for linha in f:`
`  nome, nota = linha.split(";")`
	Ler linha a linha e partir cada uma.

`f.read()  f.readlines()`	Tudo num texto · lista de linhas.

Python essencial – consulta rápida

15 CLASSES E OBJETOS

cap. 20

```
class Conta:
    def __init__(self, nome, saldo=0):
        self.titular = nome
        self.saldo = saldo

    def depositar(self, valor):
        self.saldo += valor

    def __str__(self):
        return f"Conta de
{self.titular}"
```

`__init__` corre ao criar o objeto. `self` é o próprio objeto. `__str__` define o que `print` mostra.

`c = Conta("Ana", 100)`	Criar um objeto (instância).
`c.depositar(50)  c.saldo`	Chamar um método · ler um atributo.

`class Poupanca(Conta):`
`  def juros(self, taxa):`
`  return self.saldo * taxa`
	Herança: reaproveita tudo de `Conta` e acrescenta.

`@dataclass`
`class Ponto:`
`  x: float`
`  y: float = 0`
	Classe de dados: o `__init__` é criado sozinho (`from dataclasses import dataclass`).

16 MÓDULOS E PACOTES

cap. 21

`import math  math.sqrt(16)`	Importar o módulo e usar com o prefixo.
`from random import randint`	Importar só um nome.
`import datetime as dt`	Dar um nome curto.
`if __name__ == "__main__":`	Só corre quando o ficheiro é executado diretamente.
`%pip install requests`	Instalar num notebook (no terminal: `pip install`).

! ARMADILHAS FREQUENTES

× errado · ✓ certo

Atribuir ou comparar. = guarda um valor; == compara.

```
✗ if x = 5:
✓ if x == 5:
```

O input devolve texto. Converte antes de fazer contas.

```
✗ idade = input("Idade: ") + 1
✓ idade = int(input("Idade: ")) + 1
```

Texto mais número. Dá `TypeError`.

```
✗ "Idade: " + 23
✓ f"Idade: {23}"
```

Copiar uma lista. Com =, as duas variáveis mudam juntas.

```
✗ b = a
✓ b = a.copy()
```

Métodos que devolvem None. `sort` altera a lista e não devolve nada.

```
✗ nums = nums.sort()
✓ nums.sort()    ou    nums = sorted(nums)
```

O fim do range não entra. Para contar de 1 a 10:

```
✗ range(1, 10)
✓ range(1, 11)
```

Comparar decimais. `0.1 + 0.2` dá `0.30000000000000004`.

```
✗ 0.1 + 0.2 == 0.3
✓ math.isclose(0.1 + 0.2, 0.3)
```

Lista como valor por omissão. É partilhada entre chamadas.

```
✗ def juntar(x, lista=[]):
✓ def juntar(x, lista=None):
```

Tuplo com um elemento. Sem vírgula é só um número.

```
✗ t = (5)
✓ t = (5,)
```